Decorator Design Pattern

1. Classification, Intent & Exam Triggers

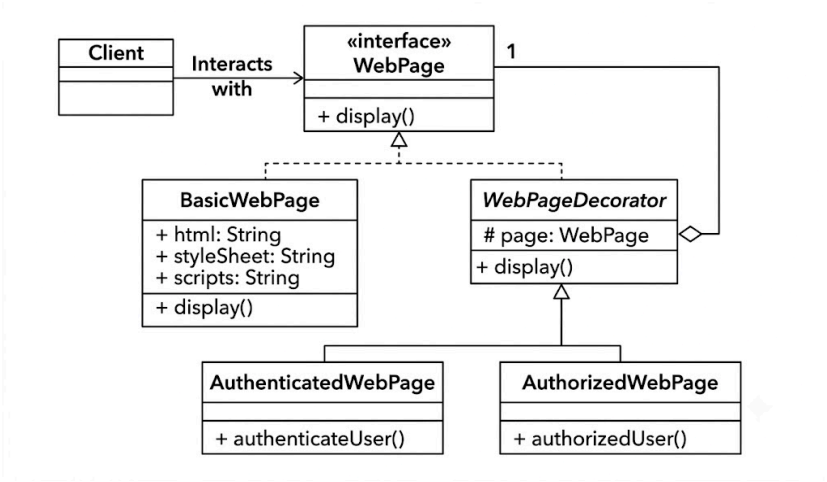
- Category: Structural Design Pattern
- Intent: Dynamically attaches additional responsibilities (behaviors or properties) to an individual object at runtime without modifying its base class or affecting other instances of the same class.
- Primary Problem Solved (Class Explosion): Eliminates exponential subclass proliferation (2^n combinations for n optional features — e.g., 3 features = 8 classes) caused by using inheritance for optional, combinable features.
- Core OOP Principles: Applies Composition over Inheritance ('has-a' wrapper reference instead of deep subclass trees) and the Open/Closed Principle (classes are open for extension via new decorators, but closed for modification so existing tested code remains untouched).
- How to Spot in Exam Questions: 'An app has a basic class (e.g., UserProfile, Coffee, Pizza, WebPage) and wants to add extra optional features over time... Instead of modifying the class directly or creating multiple subclasses, which pattern should be used?'

2. Pattern Structure, Participants & UML Relationships

Participant Role	Responsibility in Pattern	Lecture & Quiz Examples
1. Component (Interface)	Defines the common interface for both the core object and all decorators so they can be used interchangeably.	WebPage, Coffee, Pizza, UserProfile
2. ConcreteComponent	The base class implementing Component; defines the core object to which optional responsibilities are attached.	BasicWebPage, Espresso, PizzaImpl, BasicUserProfile
3. Decorator (Abstract Class)	Implements Component AND holds a protected reference (has-a composition/aggregation) to a Component object; forwards requests to it.	WebPageDecorator, CoffeeDecorator, PizzaDecorator, ProfileDecorator
4. ConcreteDecorator	Extends Decorator to add specific state or extra behavior before or after delegating to the wrapped Component.	AuthorizedWebPage, MilkDecorator, Cheese, PremiumBadgeDecorator

UML Relationships

- implements (Realization — dashed line, hollow triangle): Both ConcreteComponent and Decorator implement the Component interface so wrapped objects share the exact same type as the base object.
- has-a (Composition / Aggregation — solid line with diamond on Decorator): Decorator holds a reference (protected Component component) to the wrapped Component object. [Exam Fact: Composition ('has-a') is the most important relationship in the Decorator Pattern].
- extends (Generalization — solid line, hollow triangle): ConcreteDecorator subclasses inherit from the abstract Decorator class.



3. Implementation Code Structure

```
//Component Interface
public interface WebPage {
   public void display();
}

public class BasicWebPage implements WebPage {
   private String html = "...";
   private String styleSheet = "...";
   private String scripts = "...";

   @Override
   public void display() {
      System.out.println("Basic web page");
   }
}
```

```
// Step 3: Abstract Decorator Class
public abstract class WebPageDecorator implements WebPage {
    protected WebPage page; // Composition (has-a relationship)

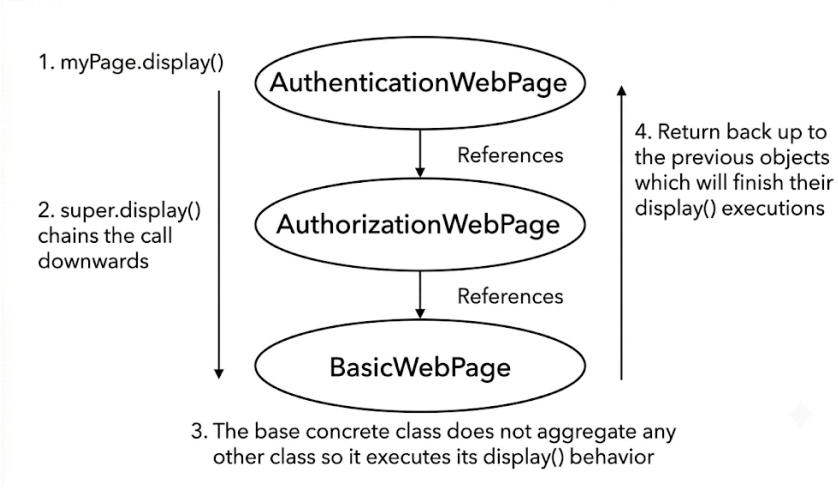
    public WebPageDecorator(WebPage webpage) {
        this.page = webpage;
    }

    @Override
    public void display() {
        this.page.display(); // Delegate to wrapped object
    }
}

// Step 4: Concrete Decorators
public class AuthorizedWebPage extends WebPageDecorator {
    public AuthorizedWebPage(WebPage decoratedPage) {
        super(decoratedPage);
    }
    public void authorizedUser() {
        System.out.println("Authorizing user");
    }
    @Override
    public void display() {
        super.display();
        this.authorizedUser(); // Added behavior
    }
}

// Step 5: Client Execution & Chaining
public class Program {
    public static void main(String[] args) {
        WebPage myPage = new BasicWebPage();
        myPage = new AuthorizedWebPage(myPage);
        myPage = new AuthenticatedWebPage(myPage);
        myPage.display();
    }
}
```

4. Execution Mechanism



5. Key Comparisons, Trade-Offs & Real-World Challenges

Decorator vs. Inheritance & Decorator vs. Proxy

Comparison Area	Decorator Pattern	Inheritance / Proxy Counterpart
vs. Inheritance (Mechanism)	Wraps objects at runtime using Composition (has-a); creates objects dynamically.	Creates new classes at compile time (is-a); causes class explosion (2^n subclasses) and rigid hierarchies.
vs. Proxy (Primary Intent)	Adds responsibilities and expands capabilities dynamically (enhancement).	Controls access to an object (security/protection, lazy initialization, remote access).
vs. Proxy (Stacking & Focus)	Stacked multiple times in flexible chains; modifies/extends what the object does.	Usually a single layer of control; acts as a surrogate/placeholder governing how the object is used.

Advantages, Disadvantages

- **Key Advantages:**

  (1) Open/Closed Principle: Existing tested structure is untouched, avoiding regression bugs.

  (2) Runtime Flexibility: Add/combine behaviors dynamically at runtime.

  (3) Incremental Development: No need to predict all features upfront during initial design.

  (4) Reusable Single-Responsibility Classes: Small, focused decorators can be reused.

- **When to Use:** When optional, combinable responsibilities must be added dynamically at runtime, or when subclassing causes class explosion.
- **When NOT to Use:** When the core behavior changes frequently, or when added features are mandatory (not optional) for all instances.

Real-World Challenges

Challenge Aspect	How to Explain
Code Complexity	Creates many small, structurally similar decorator classes (one for every optional feature), increasing package clutter and system complexity.
Debugging	Because a single logical object is wrapped across multiple runtime layers, tracing calls through deep super.operation() chains and inspecting variables in a debugger is difficult.
Performance	Each wrapper creates an additional object in memory and adds an extra method-delegation hop on every call, increasing memory usage and minor execution overhead when deeply stacked.
Readability	Nested constructor chaining (new Theme(new TwoFactor(new Badge(new BasicProfile())))) makes client code verbose and hard to read (often requires a Builder or Factory to clean up).
Order Dependency & Conflicts	The order in which decorators are wrapped matters (e.g., authenticating before authorizing, or compressing before encrypting). Applying decorators in the wrong sequence causes conflicts or bugs.